

Lab6 - APIs REST y Silence

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

Índice

1. Objetivo	2
1.1. 1. Inicialización del Proyecto y Configuración del Entorno	3
1.2. 2. Creación de Endpoints (GET y POST con Parámetros)	4
1.3. 3. Pruebas de Endpoints y Autenticación Automática	7
1.4. 4. Sistema de Usuarios y Roles	10
1.5. 5. La Consola Integrada	11
2. Ejercicios de Generación de Endpoints y Pruebas en Silence	13
3. Ejercicio 1: Asignaturas de un Grado	15
4. Ejercicio 2: Alumnos Matriculados en una Asignatura	17
5. Ejercicio 3: Nota Media de una Asignatura	19
6. Ejercicio 4: Alumnos con Matrícula de Honor	21
7. Ejercicio 5: Carga Docente por Profesor	23
8. Ejercicio 6: Grupos de una Asignatura en un Año Académico	25
9. Ejercicio 7: Búsqueda de Alumnos por Apellido	27
10. Ejercicio 8: Alumnos con más de N Asignaturas Matriculadas	29
11. Ejercicio 9: Crear Nota con Validación de Reglas de Negocio	31
12. Ejercicio 10 (Avanzado): Mis Calificaciones (uso de user_id)	33
13. Ejercicio 11 (Avanzado): Matriculación con Restricciones de Rol	35
14. Ejercicio 12 (Avanzado): Uso combinado de {param} y user_id 	37
14.1. Notas	38

i Info

Versión PDF disponible

Objetivo

El objetivo de esta práctica es usar el framework de backend Silence para implementar una API RESTful con la que acceder y modificar los elementos existentes en una base de datos relacional. El alumno aprenderá a:

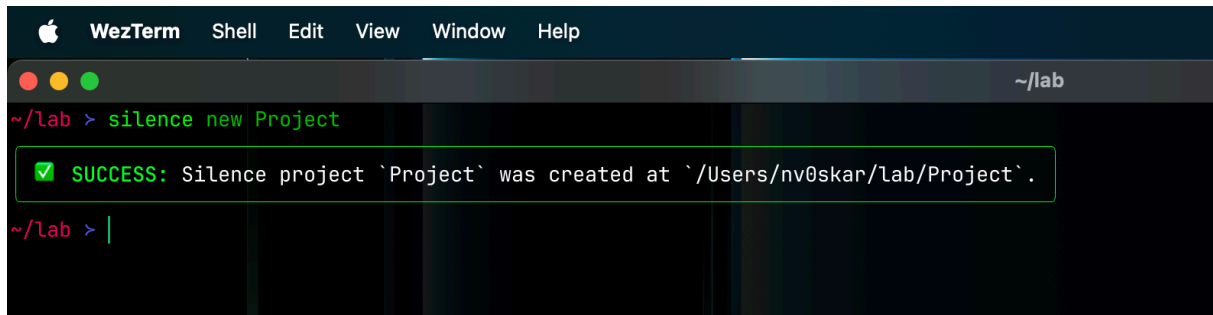
- Instalar el framework Silence
- Crear y configurar un proyecto Silence
- Crear los endpoints asociados a la base de datos
- Realizar pruebas sobre la API RESTful creada.

Silence constituye un framework de propósito educacional desarrollado en la Universidad de Sevilla, diseñado para simplificar la construcción de APIs RESTful y aplicaciones web a partir de una base de datos relacional. Su instalación puede efectuarse a través de su repositorio oficial en GitHub o, en macOS/LinuxOSs, mediante el gestor de paquetes Homebrew ejecutando el comando `$ brew install IISSI-US/tap/silence`.

A continuación, se presenta un tutorial exhaustivo y formal sobre el uso de Silence, abarcando desde la configuración inicial hasta la gestión avanzada de endpoints, pruebas y usuarios.

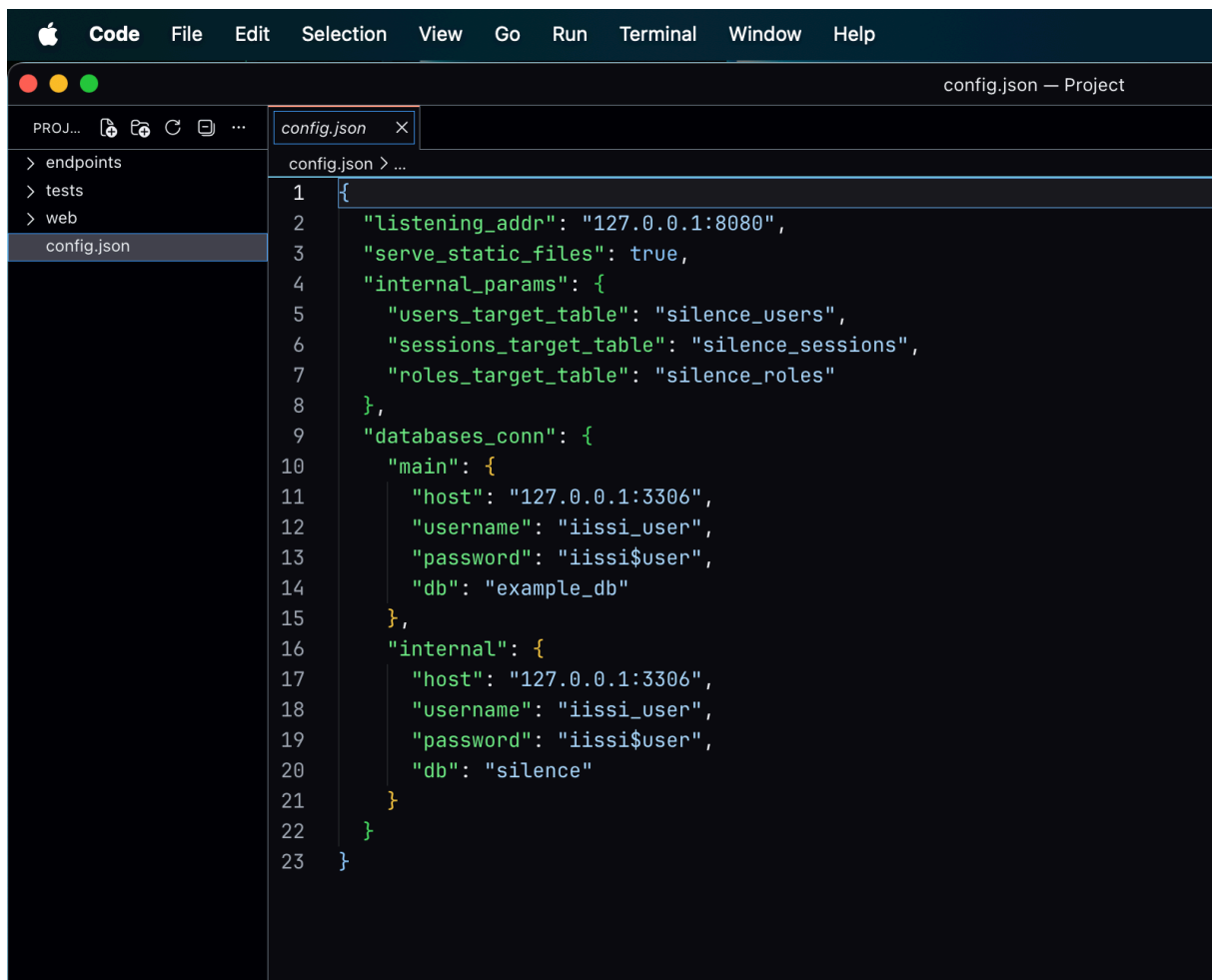
1.1. 1. Inicialización del Proyecto y Configuración del Entorno

El primer paso consiste en la creación de un nuevo proyecto. Para ello, se debe abrir una terminal y ejecutar el comando `$ silence new Project`, el cual generará la estructura de directorios necesaria y confirmará la creación exitosa del proyecto.



```
Apple WezTerm Shell Edit View Window Help
~/lab
~/lab > silence new Project
[✓] SUCCESS: Silence project `Project` was created at `/Users/nv0skar/lab/Project`.
~/lab > |
```

Posteriormente, es fundamental configurar los parámetros del servidor. En el archivo `config.json`, se definen aspectos cruciales como la dirección de escucha (`listening_addr`), la configuración de archivos estáticos (`serve_static_files`), los parámetros internos para las tablas de usuarios, sesiones y roles, así como las conexiones a las bases de datos principal e interna (`host`, `usuario`, `contraseña` y `nombre de la base de datos`).



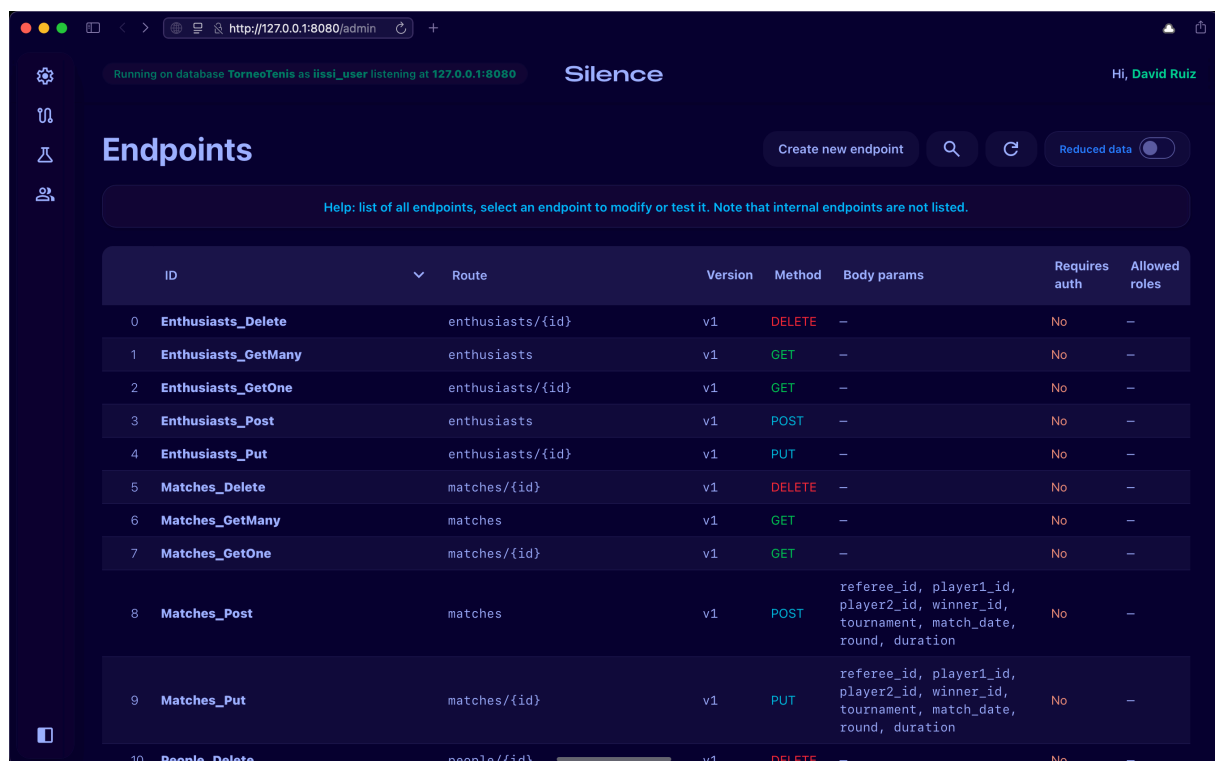
```
Apple Code File Edit Selection View Go Run Terminal Window Help
config.json — Project
config.json
> endpoints
> tests
> web
config.json
1 {
2   "listening_addr": "127.0.0.1:8080",
3   "serve_static_files": true,
4   "internal_params": {
5     "users_target_table": "silence_users",
6     "sessions_target_table": "silence_sessions",
7     "roles_target_table": "silence_roles"
8   },
9   "databases_conn": {
10    "main": {
11      "host": "127.0.0.1:3306",
12      "username": "iissi_user",
13      "password": "iissi$user",
14      "db": "example_db"
15    },
16    "internal": {
17      "host": "127.0.0.1:3306",
18      "username": "iissi_user",
19      "password": "iissi$user",
20      "db": "silence"
21    }
22  }
23 }
```

Una vez configurado el entorno, se procede a iniciar el servidor local ejecutando el comando `$ silence run ~/Project`. El sistema desplegará la información de conexión a las bases de datos y quedará a la espera de peticiones en la dirección especificada (por defecto, `http://127.0.0.1:8080`).

```
~/lab/Project > silence run
2026-09-17T07:42:51.528273Z INFO waveless_sql::databases::mysql: Creating new MySQL database connection (127.0.0.1:3306) on TorneoTennis.
2026-09-17T07:42:51.615667Z INFO load: waveless_commons::databases: Creating main's pool.
2026-09-17T07:42:51.615762Z INFO load: waveless_sql::databases::mysql: Creating new MySQL database connection (127.0.0.1:3306) on TorneoTennis.
2026-09-17T07:42:51.680230Z INFO load: waveless_commons::databases: Creating internal's pool.
2026-09-17T07:42:51.680305Z INFO load: waveless_sql::databases::mysql: Creating new MySQL database connection (127.0.0.1:3306) on silence.
⚡ Silence server running at http://127.0.0.1:8080
You can access the admin panel from http://127.0.0.1:8080/admin
2026-09-17T07:42:51.892865Z INFO serve: waveless_runtime::server: Executing 'Silence App' on 127.0.0.1:8080 at 2026-09-17 09:42:51.892668 +02:00
```

1.2. 2. Creación de Endpoints (GET y POST con Parámetros)

La gestión de endpoints se realiza a través de la interfaz administrativa. En la sección “Endpoints”, se despliega un listado de todos los puntos de acceso disponibles, detallando su ID, ruta, versión, método HTTP, parámetros del cuerpo, si requieren autenticación y los roles permitidos. Cabe destacar que los endpoints internos no se listan en esta vista.



Help: list of all endpoints, select an endpoint to modify or test it. Note that internal endpoints are not listed.

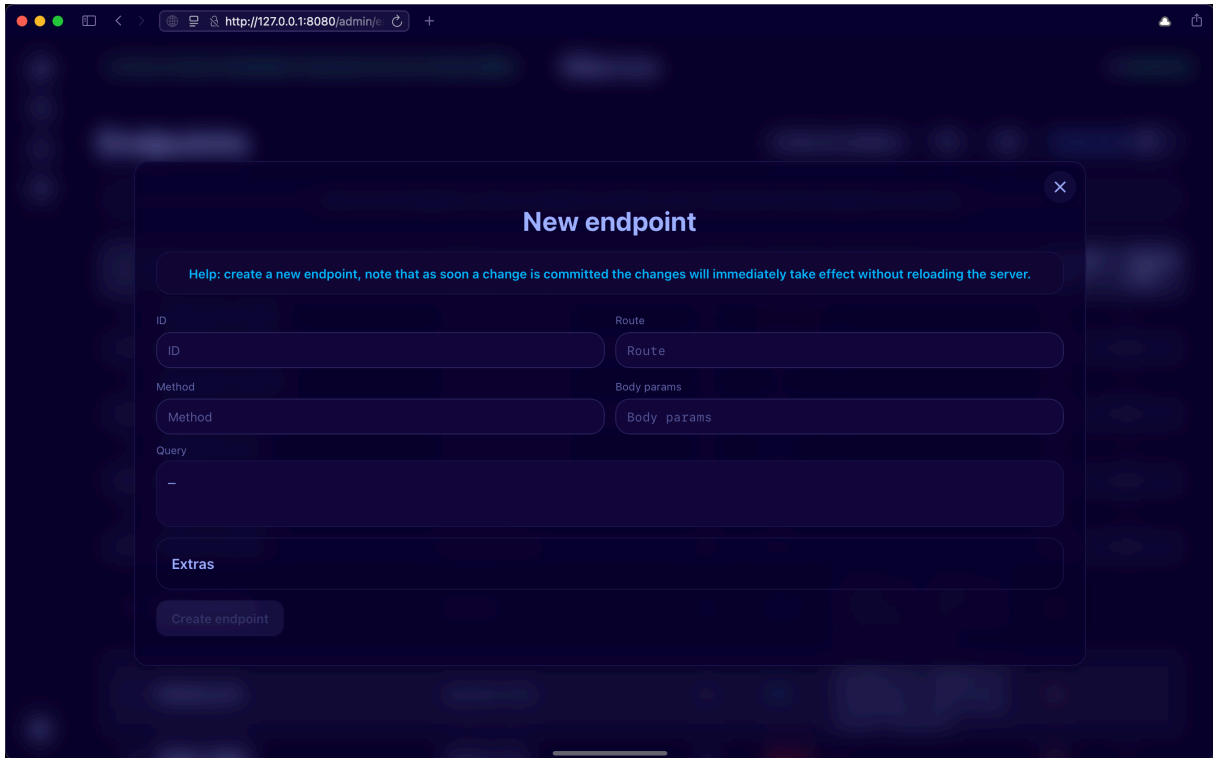
ID	Route	Version	Method	Body params	Requires auth	Allowed roles
0	enthusiasts/{id}	v1	DELETE	—	No	—
1	enthusiasts	v1	GET	—	No	—
2	enthusiasts/{id}	v1	GET	—	No	—
3	enthusiasts	v1	POST	—	No	—
4	enthusiasts/{id}	v1	PUT	—	No	—
5	matches/{id}	v1	DELETE	—	No	—
6	matches	v1	GET	—	No	—
7	matches/{id}	v1	GET	—	No	—
8	matches	v1	POST	referee_id, player1_id, player2_id, winner_id, tournament, match_date, round, duration	No	—
9	matches/{id}	v1	PUT	referee_id, player1_id, player2_id, winner_id, tournament, match_date, round, duration	No	—
10	people/{id}	v1	DELETE	—	No	—

Para crear un nuevo endpoint, se selecciona la opción “Create new endpoint”. El sistema permite definir endpoints que surtirán efecto inmediatamente sin necesidad de reiniciar el servidor.

1. Objetivo

Info

Nota: se pueden concatenar queries SQL usando ; (e.g.: al crear un nuevo alumno se crea un nuevo usuario Silence con un rol asociado).

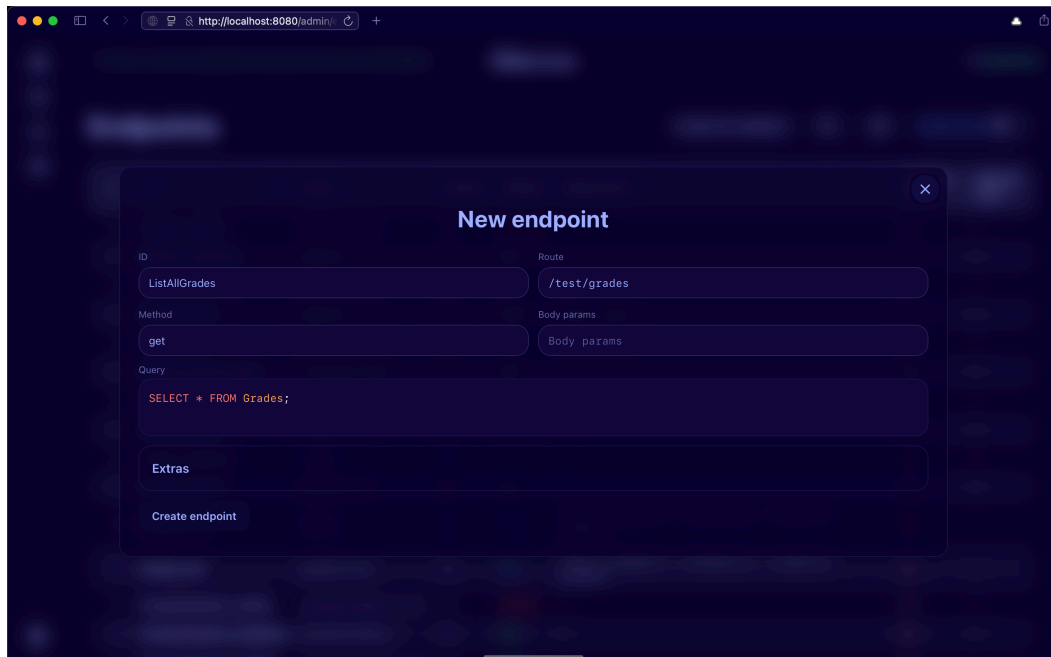


The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:8080/admin/`. A modal titled "New endpoint" is open, featuring a close button (X) in the top right corner. Below the title, a help message states: "Help: create a new endpoint, note that as soon a change is committed the changes will immediately take effect without reloading the server." The form contains several input fields: "ID" (with a placeholder "ID"), "Route" (with a placeholder "Route"), "Method" (with a placeholder "Method"), and "Body params" (with a placeholder "Body params"). There is also a "Query" field with a placeholder "-". Below these fields is an "Extras" section. At the bottom of the modal is a "Create endpoint" button.

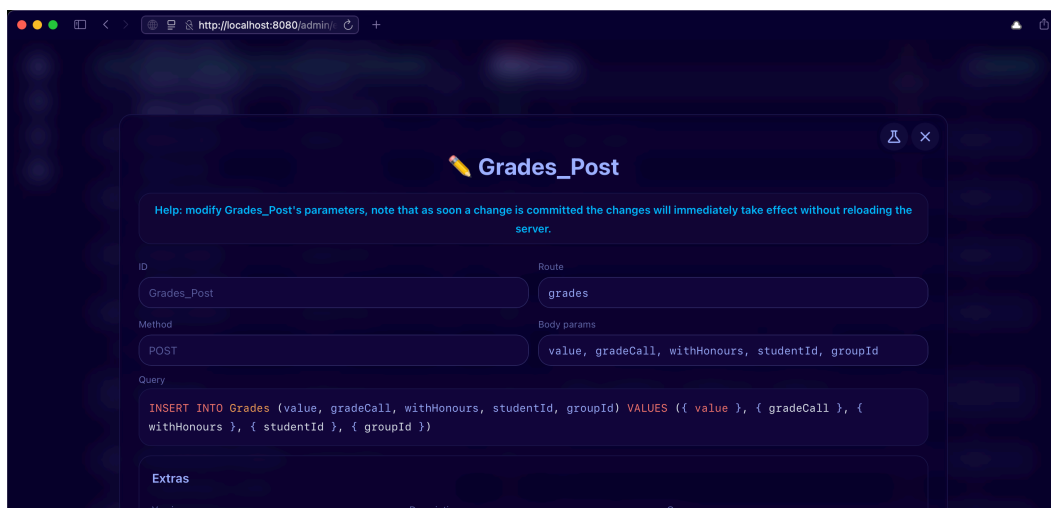
i Info

Ejemplos:

- **Endpoint GET:** Para definir una operación de lectura, se completan los campos con el identificador (`ListAllGrades`), la ruta (`/test/grades`), el método (`get`) y la consulta SQL asociada (`SELECT * FROM Grades ;`).

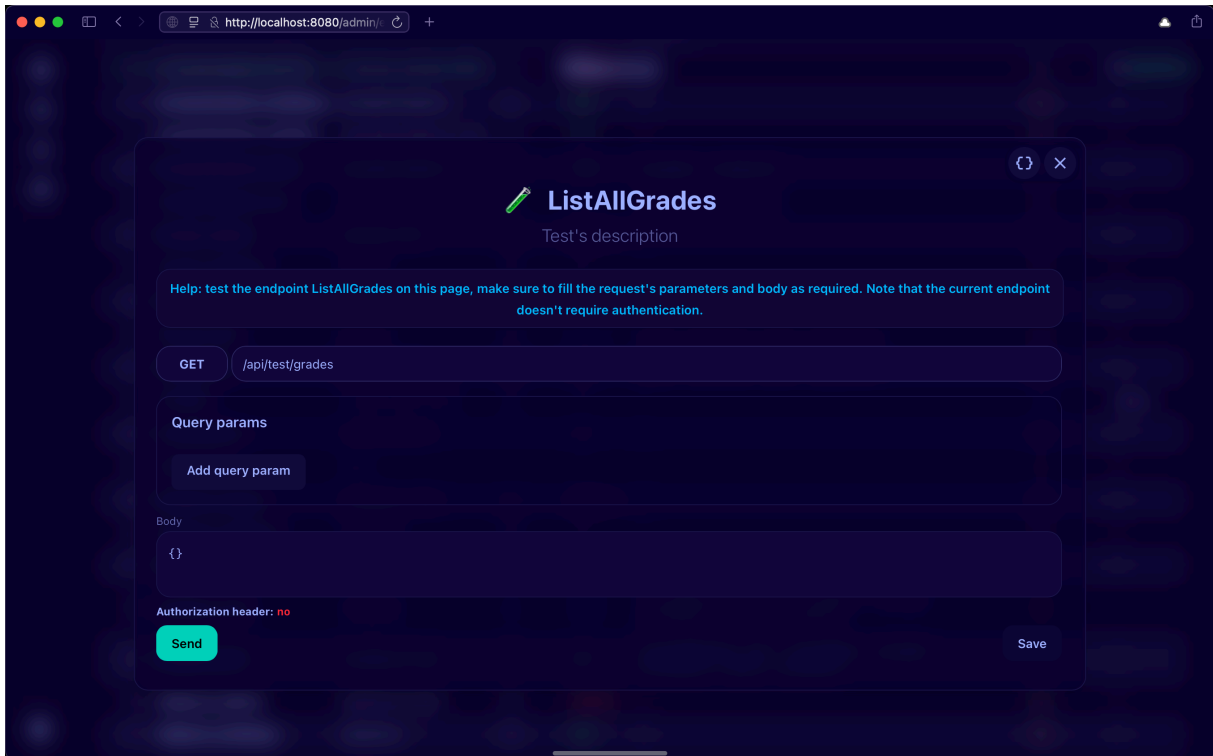


- **Endpoint POST con Parámetros:** Para operaciones de inserción, se define el ID (`Grades_Post`), la ruta (`grades`), el método (`POST`), los parámetros del cuerpo (`value`, `gradeCall`, `withHonours`, `studentId`, `groupId`) y la consulta SQL de inserción correspondiente. Adicionalmente, en la sección “Extras” se puede especificar la versión, descripción, parámetros de consulta, roles permitidos y ruta.

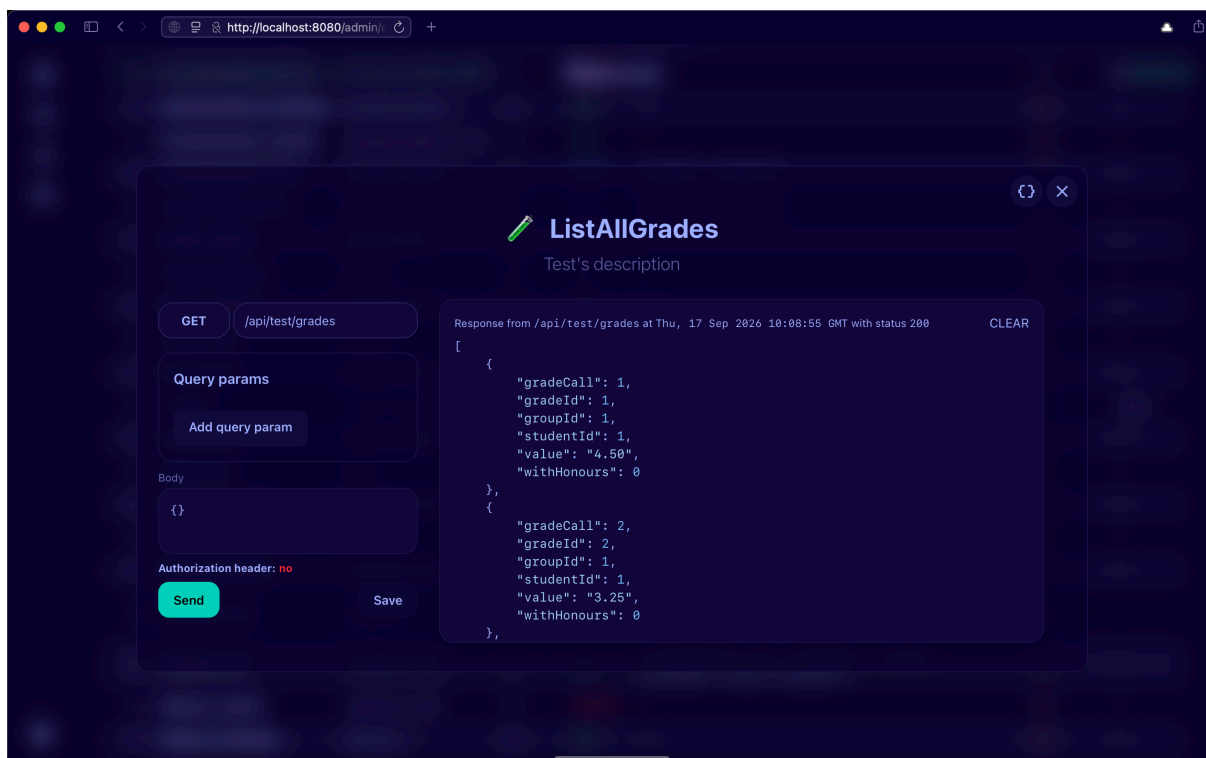


1.3. 3. Pruebas de Endpoints y Autenticación Automática

Silence proporciona una interfaz integrada para probar los endpoints de manera ágil. Al seleccionar un endpoint, como `Grades_GetMany`, se despliega un panel que permite configurar y enviar peticiones HTTP.

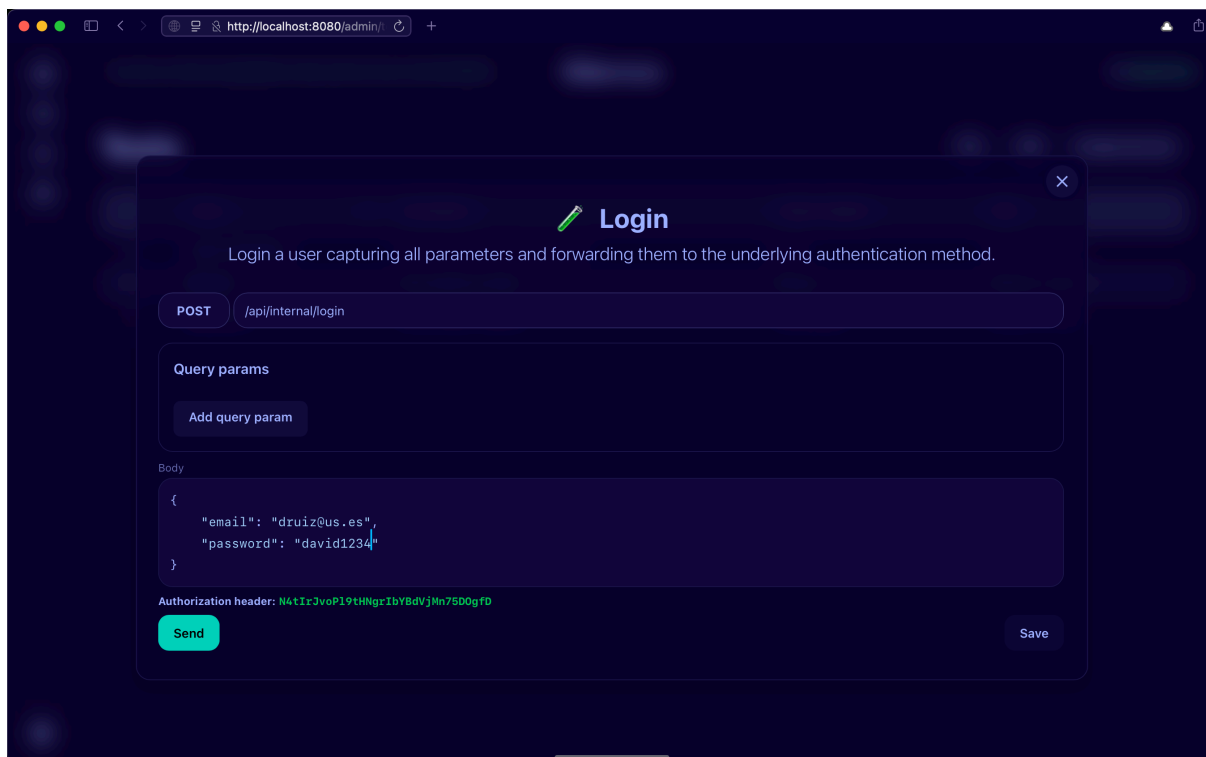


Al pulsar “Send”, el servidor procesa la solicitud y devuelve la respuesta en formato JSON, visualizándose los datos recuperados. Es posible observar que, para endpoints públicos, el campo “Authorization header” indica “no”.



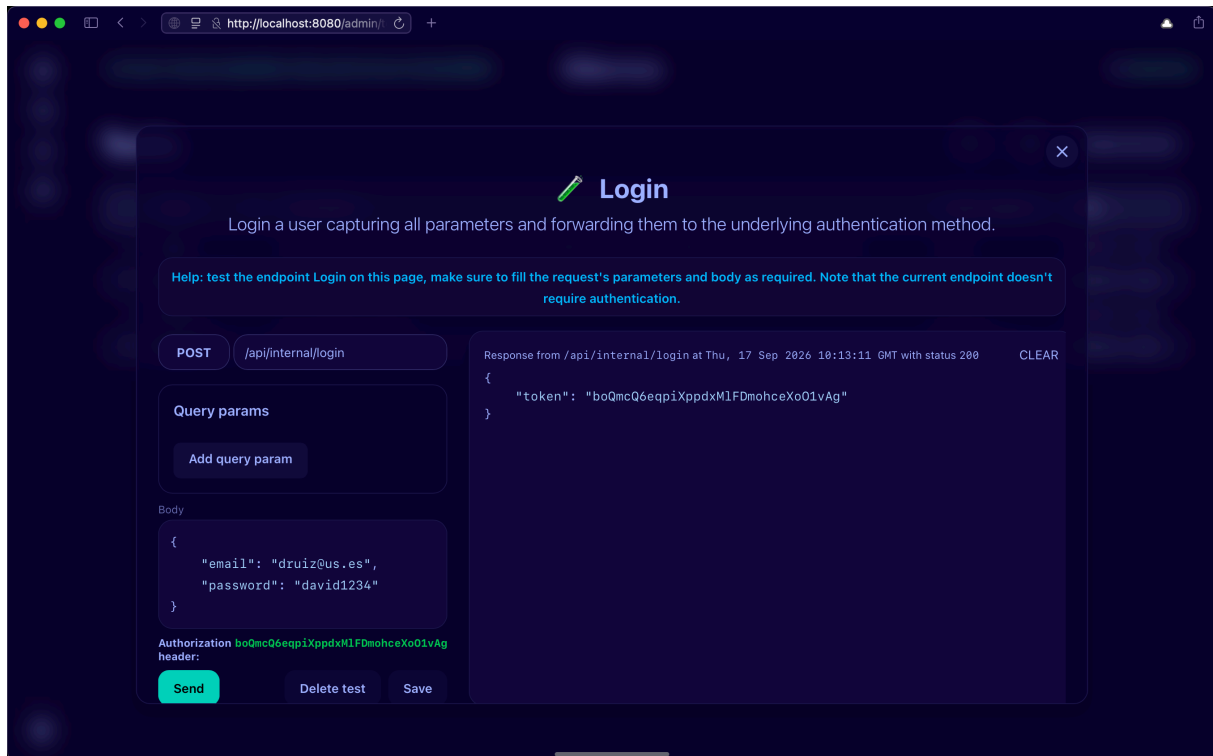
Para interactuar con endpoints protegidos, es necesario autenticarse. Esto se logra atacando el endpoint de Login (`/api/internal/login`).

Al enviar una petición POST con las credenciales válidas en el cuerpo (email y contraseña), el servidor responde con un token de autorización. La interfaz de Silence captura automáticamente este token y lo almacena en el campo “Authorization header”.



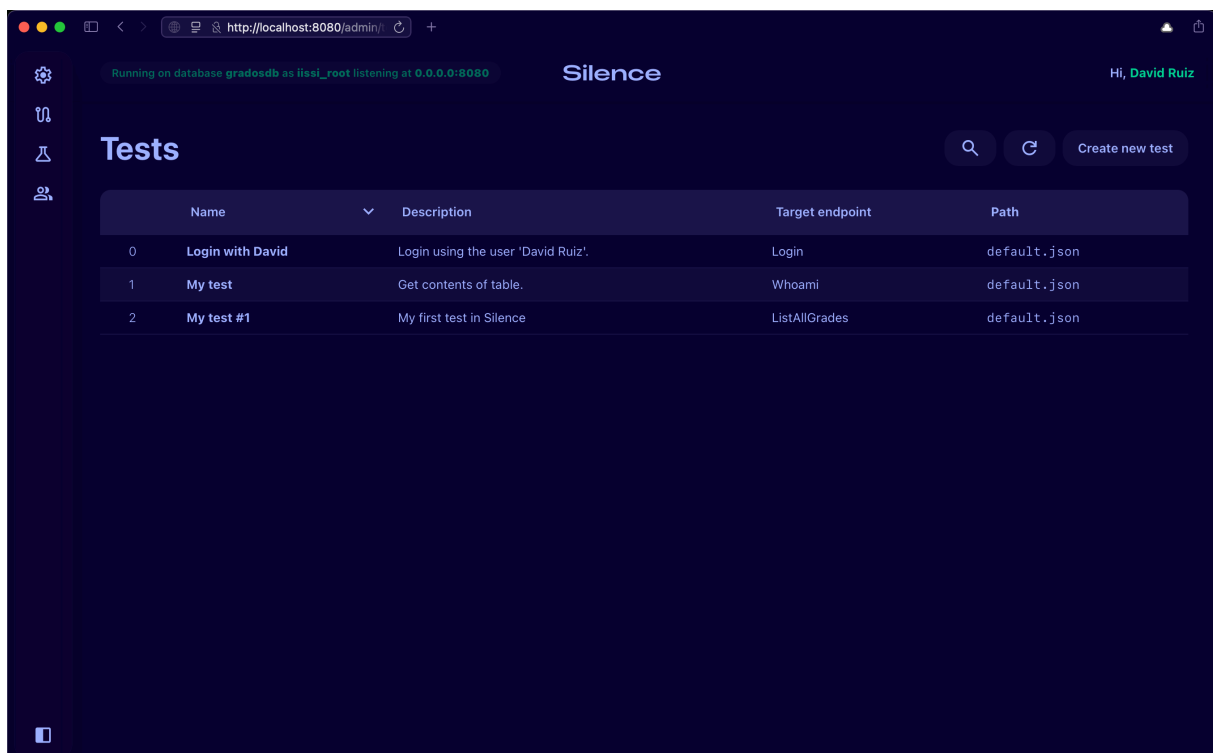
1. Objetivo

De este modo, las peticiones subsecuentes incluirán dicho token de forma automática, facilitando enormemente el flujo de trabajo.



Asimismo, es posible guardar las pruebas para su reutilización. Al configurar una petición, se puede hacer clic en “Save” y asignarle un título y una descripción.

Posteriormente, en la sección “Tests”, se podrá acceder a un listado de todas las pruebas almacenadas, identificadas por su nombre, descripción, endpoint objetivo y ruta.



1.4. 4. Sistema de Usuarios y Roles

La administración de usuarios se gestiona desde la sección “Users”. En esta vista se muestra un listado de los usuarios registrados, detallando su ID, nombre, correo electrónico, rol y contraseña (debidamente ofuscada). Los roles permiten segmentar los permisos, distinguiendo entre administradores (`admin`), mantenedores (`maintain`), probadores (`tester`) o usuarios sin rol asignado. En este contexto, en `GradesDB` usaremos los roles: `admin` , `profesor` , `alumno` .

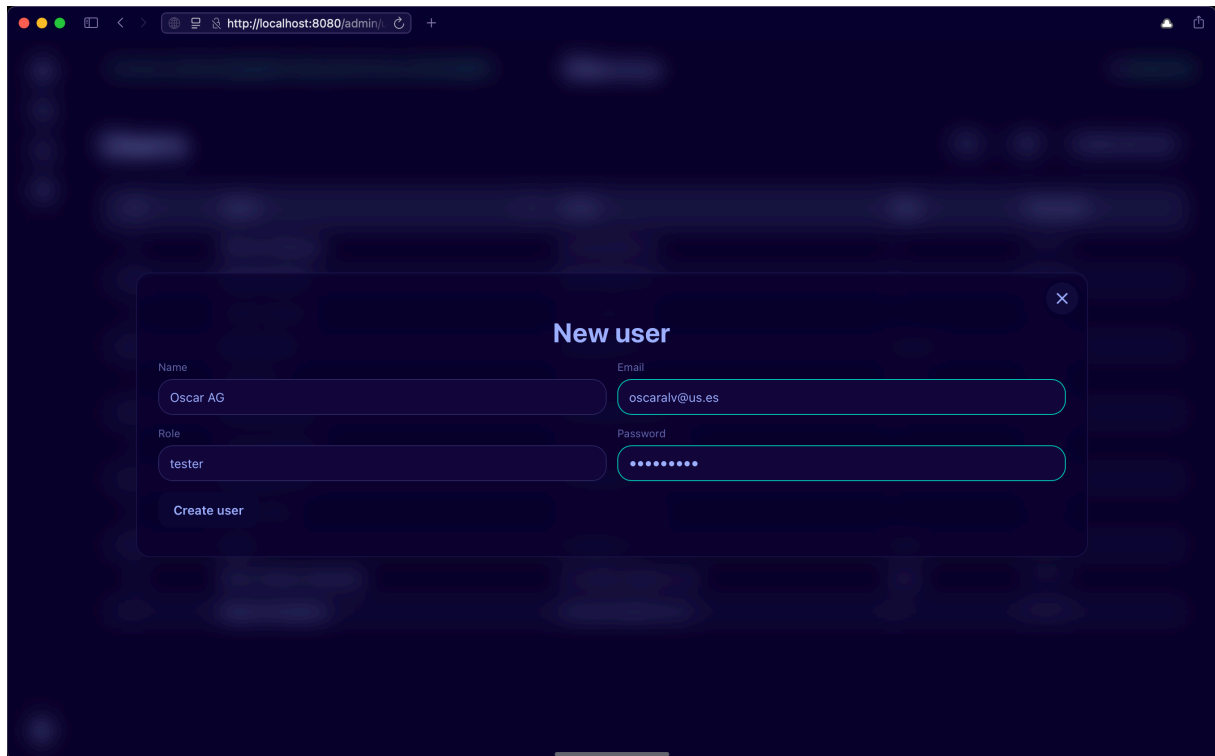
i Info

También es posible añadir roles directamente desde *HeidiSQL*. Conéctese a la base de datos `silence` , y ejecute el siguiente query (reemplazando `user_id` por el id de usuario que se ha creado anteriormente y `role_name` por su rol):

```
INSERT INTO silence_roles(user_id, role) VALUES (user_id,
'profesor');
```

ID	Name	Email	Role	Password
5	Alfonso Márquez	amarquez@us.es	—	*****
10	Andrea Gómez	agomez@mail.es	—	*****
2	Carlos Arévalo	carevalo@us.es	—	*****
6	Daniel Ayala	dayala1@us.es	maintain	*****
1	David Ruiz	druiz@us.es	admin	*****
11	Ernesto Murillo	emurillo@correo.es	—	*****
4	Inma Hernández	inmahernandez@us.es	—	*****
3	Margarita Cruz	mcruz@us.es	—	*****
8	Marta López	mlopez@mail.com	—	*****
27	Oscar	oscar@us.es	admin	*****
40	Oscar AG	oscaralv@us.es	tester	*****
7	Raquel Sampedro	rsampedro@gmail.com	—	*****

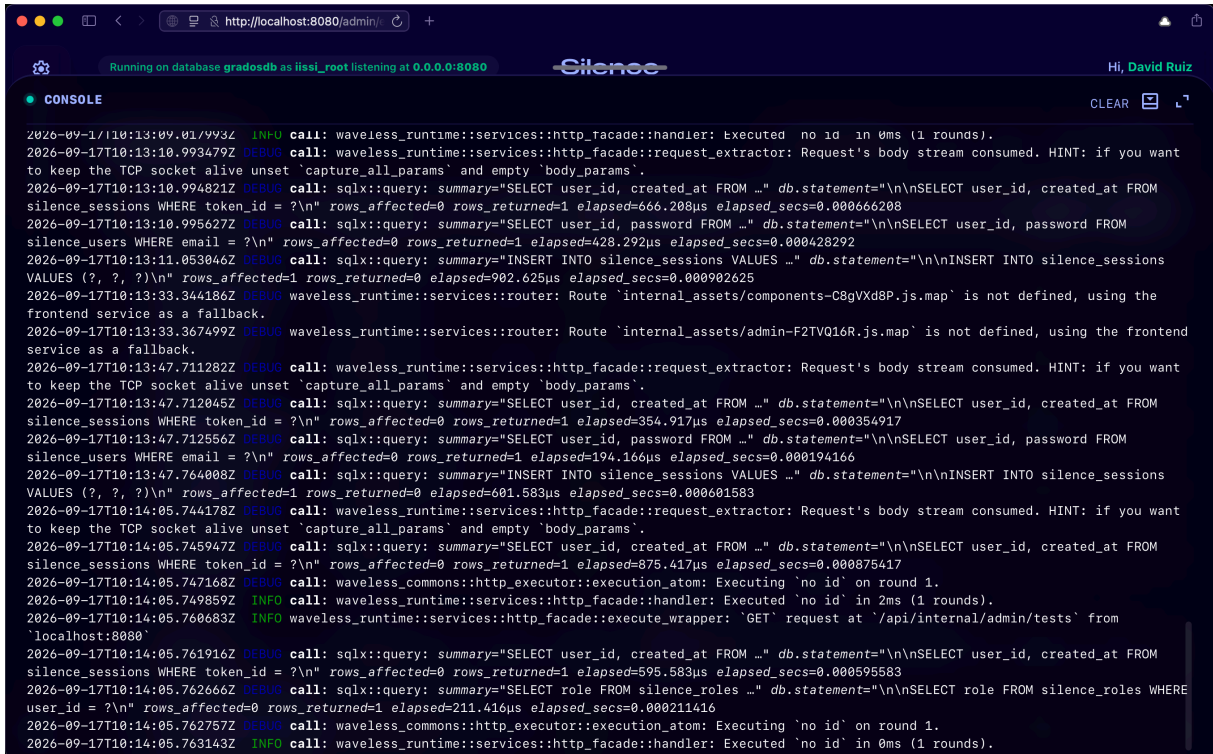
Para dar de alta a un nuevo usuario, se selecciona “Create new user” y se completa el formulario con los datos requeridos: nombre, correo electrónico, rol y contraseña.



Adicionalmente, el sistema cuenta con una página de registro público (“Sign up”) (/auth/signup). Tenga en cuenta que el primer usuario que sea registrado le será asignado el rol de administrador automáticamente.

1.5. 5. La Consola Integrada

Finalmente, la interfaz de Silence cuenta con una consola integrada en la parte inferior de la pantalla. Esta herramienta es de vital importancia para la depuración, ya que proporciona un registro detallado en tiempo real de todas las operaciones del servidor. En ella se pueden observar las peticiones HTTP entrantes, las consultas SQL ejecutadas (con su respectivo tiempo de ejecución), y diversos mensajes de depuración (INFO, DEBUG), lo que permite identificar y resolver problemas con gran precisión.



The screenshot shows the Silencio console interface. At the top, it indicates the application is running on database 'gradosdb' as 'liisi_root' listening at '0.0.0.0:8080'. The console title is 'Silencio' and the user is 'Hi, David Ruiz'. The main area is labeled 'CONSOLE' and contains a log of events. The log includes timestamps, log levels (INFO, DEBUG), and messages from various components like 'waveless_runtime::services::http_facade::handler', 'request_extractor', 'sqlx::query', and 'router'. It details database queries, their results, and execution times. For example, it shows a 'SELECT user_id, created_at FROM silence_sessions WHERE token_id = ?' query and an 'INSERT INTO silence_sessions' statement. The log also mentions route fallbacks for 'internal_assets/components-C8gVXd8P.js.map' and 'internal_assets/admin-F2TVQ16R.js.map'.

```

2026-09-17T10:13:09.017993Z INFO call: waveless_runtime::services::http_facade::handler: Executed 'no id' in 0ms (1 rounds).
2026-09-17T10:13:10.993479Z DEBUG call: waveless_runtime::services::http_facade::request_extractor: Request's body stream consumed. HINT: if you want
to keep the TCP socket alive unset 'capture_all_params' and empty 'body_params'.
2026-09-17T10:13:10.994821Z DEBUG call: sqlx::query: summary="SELECT user_id, created_at FROM ..." db.statement="\n\nSELECT user_id, created_at FROM
silence_sessions WHERE token_id = ?\n" rows_affected=0 rows_returned=1 elapsed=666.208µs elapsed_secs=0.000666208
2026-09-17T10:13:10.995627Z DEBUG call: sqlx::query: summary="SELECT user_id, password FROM ..." db.statement="\n\nSELECT user_id, password FROM
silence_users WHERE email = ?\n" rows_affected=0 rows_returned=1 elapsed=428.292µs elapsed_secs=0.000428292
2026-09-17T10:13:11.053046Z DEBUG call: sqlx::query: summary="INSERT INTO silence_sessions VALUES ..." db.statement="\n\nINSERT INTO silence_sessions
VALUES (?, ?, ?)\n" rows_affected=1 rows_returned=0 elapsed=902.625µs elapsed_secs=0.000902625
2026-09-17T10:13:33.344186Z DEBUG waveless_runtime::services::router: Route 'internal_assets/components-C8gVXd8P.js.map' is not defined, using the
frontend service as a fallback.
2026-09-17T10:13:33.367499Z DEBUG waveless_runtime::services::router: Route 'internal_assets/admin-F2TVQ16R.js.map' is not defined, using the frontend
service as a fallback.
2026-09-17T10:13:47.71282Z DEBUG call: waveless_runtime::services::http_facade::request_extractor: Request's body stream consumed. HINT: if you want
to keep the TCP socket alive unset 'capture_all_params' and empty 'body_params'.
2026-09-17T10:13:47.712045Z DEBUG call: sqlx::query: summary="SELECT user_id, created_at FROM ..." db.statement="\n\nSELECT user_id, created_at FROM
silence_sessions WHERE token_id = ?\n" rows_affected=0 rows_returned=1 elapsed=354.917µs elapsed_secs=0.000354917
2026-09-17T10:13:47.712556Z DEBUG call: sqlx::query: summary="SELECT user_id, password FROM ..." db.statement="\n\nSELECT user_id, password FROM
silence_users WHERE email = ?\n" rows_affected=0 rows_returned=1 elapsed=194.166µs elapsed_secs=0.000194166
2026-09-17T10:13:47.764008Z DEBUG call: sqlx::query: summary="INSERT INTO silence_sessions VALUES ..." db.statement="\n\nINSERT INTO silence_sessions
VALUES (?, ?, ?)\n" rows_affected=1 rows_returned=0 elapsed=601.583µs elapsed_secs=0.000601583
2026-09-17T10:14:05.744178Z DEBUG call: waveless_runtime::services::http_facade::request_extractor: Request's body stream consumed. HINT: if you want
to keep the TCP socket alive unset 'capture_all_params' and empty 'body_params'.
2026-09-17T10:14:05.745947Z DEBUG call: sqlx::query: summary="SELECT user_id, created_at FROM ..." db.statement="\n\nSELECT user_id, created_at FROM
silence_sessions WHERE token_id = ?\n" rows_affected=0 rows_returned=1 elapsed=875.417µs elapsed_secs=0.000875417
2026-09-17T10:14:05.747168Z DEBUG call: waveless_commons::http_executor::execution_atom: Executing 'no id' on round 1.
2026-09-17T10:14:05.749859Z INFO call: waveless_runtime::services::http_facade::handler: Executed 'no id' in 2ms (1 rounds).
2026-09-17T10:14:05.760683Z INFO waveless_runtime::services::http_facade::execute_wrapper: 'GET' request at '/api/internal/admin/tests' from
localhost:8080
2026-09-17T10:14:05.761916Z DEBUG call: sqlx::query: summary="SELECT user_id, created_at FROM ..." db.statement="\n\nSELECT user_id, created_at FROM
silence_sessions WHERE token_id = ?\n" rows_affected=0 rows_returned=1 elapsed=595.583µs elapsed_secs=0.000595583
2026-09-17T10:14:05.762666Z DEBUG call: sqlx::query: summary="SELECT role FROM silence_roles ..." db.statement="\n\nSELECT role FROM silence_roles WHERE
user_id = ?\n" rows_affected=0 rows_returned=1 elapsed=211.416µs elapsed_secs=0.000211416
2026-09-17T10:14:05.762757Z DEBUG call: waveless_commons::http_executor::execution_atom: Executing 'no id' on round 1.
2026-09-17T10:14:05.763143Z INFO call: waveless_runtime::services::http_facade::handler: Executed 'no id' in 0ms (1 rounds).

```

Ejercicios de Generación de Endpoints y Pruebas en Silence

A continuación, se proponen una serie de Ejercicios progresivos cuyo objetivo es afianzar el dominio de Silence para la definición de endpoints personalizados, la construcción de consultas SQL con JOINS y agregaciones, y la elaboración de pruebas de aceptación coherentes con las reglas de negocio del esquema `GradesDB`.

Es fundamental distinguir dos tipos de parámetros:

- Parámetros de petición (`{param_id}`): proceden directamente de la petición HTTP (segmentos de la ruta, query params o campos del cuerpo JSON) y se referencian en la consulta SQL entre llaves.
- Parámetros inyectados en tiempo de ejecución (`|param_id|`): los proporciona el propio framework a partir del contexto de la petición. El caso paradigmático es `|user_id|`, que Silence inyecta automáticamente a partir del token de autorización del usuario autenticado.

Cada Ejercicio incluye: el enunciado, la definición del endpoint en formato JSON, la solución SQL propuesta y la suite de pruebas asociada. Recuérdese que, en el entorno actual de Silence, las operaciones exitosas devuelven **200 OK** y las operaciones inválidas devuelven **500 Internal Server Error**.

Ejercicios de generación de endpoints y pruebas

i Info

Las siguientes definiciones de endpoints en JSON son orientativas y deben ser creados desde `/admin/endpoints/new`. Si se desea se puede añadir el endpoint directamente desde su definición serializada (en JSON) haciendo un test POST al endpoint interno `/api/internal/admin/endpoint` o añadiéndolo al directorio `/endpoints` del proyecto (requiere reiniciar el runtime).

i Info

Para poder hacer los tests relacionados con el uso de `|user_id|` es necesario realizar modificaciones a la base de datos, entre ellas añadir la siguiente relación:

```
ALTER TABLE people
ADD CONSTRAINT people_silence_users
FOREIGN KEY (person_id) REFERENCES silence.silence_users(user_id)
ON DELETE CASCADE ON UPDATE CASCADE;
```

Asegúrese de que `silence_users` tiene exactamente el mismo tipo de PRIMARY KEY s que `people` (INT UNSIGNED).

Es fundamental distinguir entre dos tipos de parámetros:

- **Parámetros de petición** (`{param}`): proceden directamente de la petición HTTP (segmentos de la ruta, *query params* o campos del cuerpo JSON) y se referencian en la consulta SQL entre llaves.
- **Parámetros inyectados en tiempo de ejecución** (`|param|`): los proporciona el propio framework a partir del contexto de la petición. El caso paradigmático es `|user_id|`, que Silence inyecta automáticamente a partir del token de autorización del usuario autenticado.

Recuérdese que las operaciones exitosas devuelven **200 OK** y las operaciones inválidas devuelven **500 Internal Server Error**.

Ejercicio 1: Asignaturas de un Grado

Enunciado. Diseñar un endpoint público que, dado el identificador de un grado (parámetro de ruta), devuelva todas sus asignaturas.

Definición (endpoints/degrees.json):

```
[
  {
    "id": "get_subjects_by_degree",
    "route": "/degrees/{degreeId}/subjects",
    "version": "v1",
    "method": "get",
    "execute": {
      "queries": [
        {
          "query": "SELECT subject_id, subject_name, acronym,
credits, course, subject_type FROM subjects WHERE degree_id = {degreeId}"
        }
      ]
    },
    "description": "Obtiene todas las asignaturas de un grado."
  }
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Código esperado
R1.1	[P]	Asignaturas de un grado existente	GET	/api/v1/degrees/1/subjects	200
R1.2	[P]	Grado sin asignaturas	GET	/api/v1/degrees/99/subjects	200 (lista vacía)
R1.3	[P]	Grado inexistente	GET	/api/v1/degrees/9999/subjects	200 (lista vacía)

Ejercicio 2: Alumnos Matriculados en una Asignatura

Enunciado. Construir un endpoint público que, dada una asignatura, devuelva los datos personales de los alumnos matriculados en ella.

Definición (`endpoints/subjects.json`):

```
[
  {
    "id": "get_students_by_subject",
    "route": "/subjects/{subjectId}/students",
    "version": "v1",
    "method": "get",
    "execute": {
      "queries": [
        {
          "query": "SELECT p.person_id, p.dni, p.first_name,
p.last_name, p.email FROM people p JOIN students s ON s.student_id =
p.person_id JOIN subject_enrollments se ON se.student_id = s.student_id
WHERE se.subject_id = {subjectId}"
        }
      ]
    },
    "description": "Obtiene los alumnos matriculados en una
asignatura."
  }
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Código esperado
R2.1	[P]	Alumnos de asignatura con matrículas	GET	/api/v1/subjects/1/students	200
R2.2	[P]	Asignatura sin alumnos	GET	/api/v1/subjects/99/students	200 (lista vacía)

Ejercicio 3: Nota Media de una Asignatura

Enunciado. Implementar un endpoint público que devuelva la nota media de todas las calificaciones de una asignatura concreta.

Definición (`endpoints/subjects.json`):

```
[
  {
    "id": "get_average_grade",
    "route": "/subjects/{subjectId}/average-grade",
    "version": "v1",
    "method": "get",
    "execute": {
      "queries": [
        {
          "query": "SELECT AVG(g.grade_value) AS average_grade,
COUNT(*) AS total_grades FROM grades g JOIN groups gr ON gr.group_id =
g.group_id WHERE gr.subject_id = {subjectId}"
        }
      ]
    },
    "description": "Obtiene la nota media de una asignatura."
  }
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Código esperado
R3.1	[P]	Nota media con calificaciones	GET	/api/v1/subjects/1/average-grade	200
R3.2	[P]	Asignatura sin calificaciones	GET	/api/v1/subjects/99/average-grade	200 (average_grade:) null

Ejercicio 4: Alumnos con Matrícula de Honor

Enunciado. Crear un endpoint público que devuelva todos los alumnos con matrícula de honor, incluyendo el nombre de la asignatura y la nota.

Definición (endpoints/grades.json):

```
[
  {
    "id": "get_honors_students",
    "route": "/grades/honors",
    "version": "v1",
    "method": "get",
    "execute": {
      "queries": [
        {
          "query": "SELECT p.first_name, p.last_name,
s.subject_name, g.grade_value, g.exam_call FROM grades g JOIN students st
ON st.student_id = g.student_id JOIN people p ON p.person_id =
st.student_id JOIN groups gr ON gr.group_id = g.group_id JOIN subjects s ON
s.subject_id = gr.subject_id WHERE g.with_honors = 1"
        }
      ]
    },
    "description": "Obtiene los alumnos con matrícula de honor."
  }
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Código esperado
R4.1	[P]	Alumnos con matrícula de honor	GET	/api/v1/grades/honors	200

Ejercicio 5: Carga Docente por Profesor

Enunciado. Diseñar un endpoint público que devuelva, por profesor, la suma total de créditos impartidos, ordenado de mayor a menor.

Definición (endpoints/professors.json):

```
[
  {
    "id": "get_workload",
    "route": "/professors/workload",
    "version": "v1",
    "method": "get",
    "execute": {
      "queries": [
        {
          "query": "SELECT p.person_id, p.first_name,
p.last_name, pr.category, SUM(tl.credits) AS total_credits FROM people p
JOIN professors pr ON pr.professor_id = p.person_id LEFT JOIN
teaching_loads tl ON tl.professor_id = pr.professor_id GROUP BY
pr.professor_id ORDER BY total_credits DESC"
        }
      ]
    },
    "description": "Obtiene la carga docente por profesor."
  }
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Código esperado
R5.1	[P]	Carga docente de todos los profesores	GET	/api/v1/professors/workload	200

Ejercicio 6: Grupos de una Asignatura en un Año Académico

Enunciado. Implementar un endpoint público que combine el identificador de asignatura (parámetro de ruta) y el año académico (*query parameter*).

Definición (`endpoints/subjects.json`):

```
[
  {
    "id": "get_groups_by_subject_and_year",
    "route": "/subjects/{subjectId}/groups",
    "version": "v1",
    "method": "get",
    "body_params": ["year"],
    "execute": {
      "queries": [
        {
          "query": "SELECT group_id, group_name, activity,
academic_year FROM groups WHERE subject_id = {subjectId} AND academic_year
= {year}"
        }
      ]
    },
    "description": "Obtiene los grupos de una asignatura en un año
académico."
  }
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Query params	Código esperado
R6.1	[P]	Grupos existentes para año válido	GET	/api/v1/subjects/1/groups	year=2024	200
R6.2	[P]	Sin grupos para ese año	GET	/api/v1/subjects/1/groups	year=1990	200 (lista vacía)
R6.3	[N]	Sin parámetro year	GET	/api/v1/subjects/1/groups	—	500

Ejercicio 7: Búsqueda de Alumnos por Apellido

Enunciado. Definir un endpoint público que permita buscar alumnos cuyo apellido contenga una subcadena recibida como *query parameter*.

Definición (`endpoints/students.json`):

```
[
  {
    "id": "search_by_last_name",
    "route": "/students/search",
    "version": "v1",
    "method": "get",
    "body_params": ["lastName"],
    "execute": {
      "queries": [
        {
          "query": "SELECT p.person_id, p.first_name,
p.last_name, p.email FROM people p JOIN students s ON s.student_id =
p.person_id WHERE p.last_name LIKE CONCAT('%', {lastName}, '%')"
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Query params	Código esperado
R7.1	[P]	Búsqueda con coincidencias	GET	/api/v1/students/search	lastName=Gar	200
R7.2	[P]	Búsqueda sin coincidencias	GET	/api/v1/students/search	lastName=Zzz	200 (lista vacía)
R7.3	[N]	Sin parámetro lastName	GET	/api/v1/students/search	—	500

Ejercicio 8: Alumnos con más de N Asignaturas Matriculadas

Enunciado. Crear un endpoint público que devuelva los alumnos matriculados en más de N asignaturas, con N recibido como *query parameter*.

Definición (endpoints/students.json):

```
[
  {
    "id": "get_top_enrolled",
    "route": "/students/top-enrolled",
    "version": "v1",
    "method": "get",
    "body_params": ["minSubjects"],
    "execute": {
      "queries": [
        {
          "query": "SELECT p.person_id, p.first_name,
p.last_name, COUNT(se.subject_id) AS num_subjects FROM people p JOIN
students s ON s.student_id = p.person_id JOIN subject_enrollments se ON
se.student_id = s.student_id GROUP BY s.student_id HAVING
COUNT(se.subject_id) > {minSubjects} ORDER BY num_subjects DESC"
        }
      ]
    },
    "description": "Obtiene los alumnos con más de N asignaturas
matriculadas."
  }
]
```

```
}  
]
```

Pruebas:

ID	Tipo	Descripción	Método	Ruta	Query params	Código esperado
R8.1	[P]	Alumnos con más de 2 asignaturas	GET	/api/v1/students/top-enrolled	minSubjects=2	200
R8.2	[P]	Umbral muy alto	GET	/api/v1/students/top-enrolled	minSubjects=100	200 (lista vacía)
R8.3	[N]	Sin parámetro minSubjects	GET	/api/v1/students/top-enrolled	—	500

Ejercicio 9: Crear Nota con Validación de Reglas de Negocio

Enunciado. Definir un endpoint para crear notas que respete las reglas RN01 , RN02 , RN05 , RN08 , RN11 y RN18 , requiriendo autenticación. Los campos del cuerpo (student_id , group_id , grade_value , exam_call , with_honors) se referencian entre llaves.

Definición (endpoints/grades.json):

```
[
  {
    "id": "create_grade",
    "route": "/grades/custom",
    "version": "v1",
    "method": "post",
    "create_grade": [
      "student_id",
      "group_id",
      "grade_value",
      "exam_call",
      "with_honors"
    ],
    "execute": {
      "queries": [
        {
          "query": "INSERT INTO grades (student_id, group_id, grade_value, exam_call, with_honors) VALUES ({student_id}, {group_id}, {grade_value}, {exam_call}, {with_honors})"
        }
      ]
    }
  }
]
```

```

    ]
    },
    "description": "Crea una nota aplicando todas las reglas de
negocio.",
    "require_auth": true,
    "allowed_roles": ["admin", "profesor"]
  }
]

```

Pruebas (tras un test de Login previo):

ID	Tipo	Descripción	Cuerpo	Código esperado
R9.1	[P]	Crear nota válida	{ "student_id": 1, "group_id": 1, "grade_value": 7.5, "exam_call": "Primera", "with_honors": false }	200
R9.2	[N]	Alumno no pertenece al grupo (RN02)	{ "student_id": 2, "group_id": 1, "grade_value": 8.0, "exam_call": "Primera", "with_honors": false }	500
R9.3	[N]	Valor de nota fuera de rango (RN11)	{ "student_id": 1, "group_id": 1, "grade_value": 11.0, "exam_call": "Segunda", "with_honors": false }	500
R9.4	[N]	Honor con nota < 9 (RN08)	{ "student_id": 1, "group_id": 1, "grade_value": 8.0, "exam_call": "Segunda", "with_honors": true }	500
R9.5	[N]	Convocatoria inválida (RN18)	{ "student_id": 1, "group_id": 1, "grade_value": 7.0, "exam_call": "Cuarta", "with_honors": false }	500
R9.6	[N]	Nota duplicada (RN01)	{ "student_id": 1, "group_id": 1, "grade_value": 6.0, "exam_call": "Primera", "with_honors": false }	500

Ejercicio 10 (Avanzado): Mis Calificaciones (uso de `|user_id|`)

Enunciado. Implementar un endpoint autenticado que devuelva **las calificaciones del propio usuario autenticado**, sin recibir su identificador por parámetro. Silence inyecta el identificador del usuario autenticado en la variable `|user_id|`, disponible en la consulta SQL. Este endpoint sólo requiere que el usuario esté autenticado, sin restricción adicional de rol.

Definición (`endpoints/grades.json`):

```
[
  {
    "id": "get_my_grades",
    "route": "/grades/me",
    "version": "v1",
    "method": "get",
    "execute": {
      "queries": [
        {
          "query": "SELECT s.subject_name, g.grade_value,
g.exam_call, g.with_honors FROM grades g JOIN groups gr ON gr.group_id =
g.group_id JOIN subjects s ON s.subject_id = gr.subject_id WHERE
g.student_id = |user_id|"
        }
      ]
    },
    "description": "Obtiene las calificaciones del usuario"
  }
]
```

```
autenticado.",
  "require_auth": true
}
```

Pruebas:

ID	Tipo	Descripción	Cabecera Token	Código esperado
R10.1	[P]	Consulta con token válido de un alumno con notas	Token del usuario autenticado	200
R10.2	[P]	Consulta con token válido de un alumno sin notas	Token de otro usuario	200 (lista vacía)
R10.3	[N]	Sin cabecera Token	—	401

Ejercicio 11 (Avanzado): Matriculación con Restricciones de Rol

Enunciado. Implementar un endpoint `POST /students/{studentId}/enroll` que matricule al alumno en una asignatura. El identificador del alumno se recibe como parámetro de ruta (`{studentId}`) y el de la asignatura en el cuerpo (`{subject_id}`). El endpoint exige autenticación y sólo está disponible para `admin` y `profesor`.

Definición (`endpoints/students.json`):

```
[
  {
    "id": "enroll_student",
    "route": "/students/{studentId}/enroll",
    "version": "v1",
    "method": "post",
    "body_params": ["subject_id"],
    "execute": {
      "queries": [
        {
          "query": "INSERT INTO subject_enrollments (student_id,
subject_id) VALUES ({studentId}, {subject_id})"
        }
      ]
    },
    "description": "Matricula a un alumno en una asignatura.",
  }
]
```

```

    "require_auth": true,
    "allowed_roles": ["admin", "profesor"]
  }
]

```

Pruebas:

ID	Tipo	Descripción	Cuerpo / Ruta	Código esperado
R11.1	[P]	Matriculación válida (rol autorizado)	POST /students/1/enroll · {"subject_id": 5}	200
R11.2	[N]	Matrícula duplicada	POST /students/1/enroll · {"subject_id": 1}	500
R11.3	[N]	Alumno inexistente	POST /students/9999/ enroll {"subject_id": 5}	500
R11.4	[N]	Sin cabecera Token	POST /students/1/enroll · {"subject_id": 5}	401
R11.5	[N]	Rol no autorizado (tester)	POST /students/1/enroll · {"subject_id": 5}	403

Ejercicio 12 (Avanzado): Uso combinado de `{param}` y `|user_id|`

Enunciado. Crear un endpoint autenticado que devuelva las notas del usuario autenticado en una convocatoria concreta, recibida como *query parameter*. Este Ejercicio combina un parámetro de petición (`{examCall}`) con un parámetro inyectado por el framework (`|user_id|`).

Definición (`endpoints/grades.json`):

```
[
  {
    "id": "get_my_grades_by_call",
    "route": "/grades/me/by-call",
    "version": "v1",
    "method": "get",
    "body_params": ["examCall"],
    "execute": {
      "queries": [
        {
          "query": "SELECT s.subject_name, g.grade_value,
g.exam_call FROM grades g JOIN groups gr ON gr.group_id = g.group_id JOIN
subjects s ON s.subject_id = gr.subject_id WHERE g.student_id = |user_id|
AND g.exam_call = {examCall}"
        }
      ]
    },
    "description": "Obtiene las calificaciones del usuario autenticado"
```

```

    en una convocatoria.",
    "require_auth": true
  }
]

```

Pruebas:

ID	Tipo	Descripción	Query params	Código esperado
R12.1	[P]	Consulta de convocatoria con notas	examCall=Primera	200
R12.2	[P]	Consulta de convocatoria sin notas	examCall=Extraordinaria	200 (lista vacía)
R12.3	[N]	Sin parámetro examCall	—	500
R12.4	[N]	Sin autenticación	examCall=Primera	401

14.1. Notas

- Para pruebas autenticadas, se debe de haber realizado una prueba de login previa cuyo token reutiliza automáticamente.